Check the Tree: Finding Wrong Results in PostgreSQL with Typed Derivation Trees and Commit-Directed Grammar Extension

Venu Kummari
v.kummari@iitg.ac.in
Indian Institute of Technology Guwahati
Guwahati, India

Bhuvan Chilukuti
c.bhuvan@iitg.ac.in
Indian Institute of Technology Guwahati
Guwahati, India

Gowtham Reddy
Indian Institute of Technology Guwahati
Guwahati, India

ABSTRACT

A database system that returns a wrong result raises no error and leaves no trace in its logs. Fuzzers for PostgreSQL mostly find crashes, and the oracles that detect wrong results work on small query languages of their own. In PostgreSQL 17, 18 and master, for example, the planner answers `true` for `'(,)'::int4range @> NULL`, where the strict operator must give NULL. Of 23 recent query-processing fixes in PostgreSQL master, 20 change what a query returns, but only 9 of them crash or raise an internal error. We present Arbor, a fuzzer that keeps every generated query as a typed derivation tree, so that metamorphic oracles (TLP, NoREC, expression rewrites, CTE and table fences, row-level security) can rewrite single nodes of known type at any query level, and triage reduces failures on the same trees. Plan variants and the last release serve as two more oracles. An LLM reads recent commits and proposes grammar alternatives and equivalence rewrites, which a validation gate accepts only if the server runs them, they reach the changed lines, and a rewrite never changes a result. In five one-hour trials per configuration, the typed grammar covers 36.7% of backend lines against 22.2% for random mutation, and the oracles double the plan shapes reached. We reported eleven bugs found with Arbor; nine are fixed upstream (six with the authors' patches), and Arbor rediscovers ten of them in one 150-second run (the eleventh existed on master for one day). Of 93 triaged candidates, 8 are real and 15 duplicate real bugs; the false positives come from a few classes, which we fixed. The LLM adds SQL features but no coverage, and one rewrite it proposed passed our gate although it is not an identity. On the benchmark of 23 real bugs, the arms found 2 of 20 at best. The main lesson is that oracle precision, not generator reach, limits a wrong-result fuzzer: a richer grammar also reaches the corners where two "equivalent" queries are not. Generalized with per-engine dialect overlays and optimizer-rule feedback, Arbor also tests DuckDB, SQLite, Turso and DataFusion; there it found five more bugs that we reported upstream with fixes.

CCS CONCEPTS

• **Software and its engineering** → **Software testing and debugging**; • **Information systems** → *Data management systems.*

KEYWORDS

fuzzing, database testing, test oracles, metamorphic testing, grammar-based testing, large language models, PostgreSQL

1 INTRODUCTION

Suppose you run a reporting query on PostgreSQL and it returns a row that should not be there. The query does not crash, no error is raised, and nothing in the logs says that the answer is wrong. This kind of failure, a *wrong result*, is the most harmful bug a database system can have, because the user has no way to notice it.

Fuzzers for database systems mostly look for crashes. SQLsmith [Seltenreich [n. d.]] generates random queries from the system catalog and SQUIRREL [Zhong et al. 2020] mutates SQL with coverage feedback; both detect a bug when the server crashes or an assertion fails. To find wrong results, a fuzzer needs a *test oracle* that knows the right answer. Differential testing compares two systems [Slutz 1998]; metamorphic oracles such as Ternary Logic Partitioning (TLP) [Rigger and Su 2020b], NoREC [Rigger and Su 2020a] and equivalent expression transformation [Jiang and Su 2024] compare a query with a rewritten query that must return the same rows. They rewrite the SQL text, so each tool builds its own small query generator that its oracle understands. Grammar-based generators reach deep into the planner but do not know which parts of a query can be rewritten safely; oracle-based tools know how to rewrite but cover a small part of SQL; and neither knows what changed in the code yesterday, though the planner of PostgreSQL master changes every week and most new bugs live in the newest code.

Figure 1 shows a real wrong result that Arbor found. Since PostgreSQL 17, the planner replaces `range @> elem` by the constant `true` when the range is unbounded on both sides, and by `false` when it is empty. The operator is strict, so for a NULL element the correct answer is NULL; the constant is wrong under `NOT`, in `IS NULL` tests, in `CASE`, and in outer joins, where unmatched rows carry NULLs. We reproduced it on the release 18.6 and on master. Arbor found it with a typed rewrite of the derivation tree (a scalar subquery around the range), in the first 150 test cases the rewrite ran. Of 23 recent fixes in PostgreSQL master that we put back (Section 4.6), 20 change what a query returns, and only 9 can be seen by a crash-only fuzzer. The closest work is query plan guidance [Ba and Rigger 2023], which steers a generator toward new plan shapes with SQLancer's oracles, and Fuzz4All [Xia et al. 2024], which uses a large language model (LLM) as the generator; neither connects the generator to the code that changed.

We present Arbor, a grammar-aware fuzzer for PostgreSQL built on three ideas:

(1) A *typed grammar* generates SQL as derivation trees. Every node knows its type and its place in the query (Section 3.1).
(2) *Tree-based oracles* rewrite the derivation tree, not the text: TLP and NoREC at any query level, typed expression rewrites, a row-level security check, and plan-variant and cross-version oracles (Section 3.2).

```
SELECT c,
       '(,)'::int4range @> c AS contains,
         -- must be NULL when c is NULL
       NOT ('empty'::int4range @> c) AS not_empty
         -- must be NULL when c is NULL
FROM (VALUES (NULL::int), (1)) v(c);

  c   | contains | not_empty
------+----------+----------
 NULL | t        | t          ← wrong: the planner folded the call
to a constant
 1    | t        | t
```

Figure 1: A wrong result in PostgreSQL 17, 18.6 and master that Arbor found: the planner support function for range containment returns `true` or `false` for a NULL element, where the strict operator gives NULL. No error or crash shows it; only an oracle that compares two equivalent queries does.

(3) *Commit-directed extension*: an LLM reads a recent commit and proposes grammar alternatives, seed queries and new equivalence rewrites; a validation gate accepts them only if the server runs them, they execute the lines the commit changed, and a rewrite never changes a result (Section 3.5).

Arbor then reduces every finding on its derivation tree and its tokens, localizes wrong results to a planner path or to an always-applied rewrite, and drafts the upstream report (Section 3.4).

Results. The typed grammar reaches 36.7% of backend lines in an hour against 22.2% for random mutation, with 99% valid statements against 26% (**RQ1**); the oracles double the distinct plan shapes. We reported eleven bugs found with Arbor: nine are fixed upstream (six with our patches), and Arbor rediscovers ten of them in one 150-second run (**RQ2**); the eleventh existed on master for one day only. Of 93 triaged candidates, 8 are real and 15 duplicate real bugs; the false positives come from a small number of classes. On the benchmark of 23 re-injected bugs, no arm found more than 2 of 20 in one hour (**RQ3**). The LLM extension adds SQL features but no coverage, and the gate accepted one rewrite that is not an identity (**RQ4**). The lesson is that oracle precision, not generator power, limits a wrong-result fuzzer: a generator that reaches more of SQL also reaches more corner cases where two "equivalent" queries are not equivalent.

Contributions. (1) A typed PostgreSQL grammar whose derivation trees serve generation, mutation, metamorphic rewriting and reduction (Section 3); (2) tree-based oracles with the precision of each (Section 4.5); (3) a commit-directed LLM extension with a validation gate that tests proposed rewrites on values and queries (Section 3.5); (4) a benchmark of 23 real PostgreSQL bugs put back into one server (Section 4.6); and (5) eleven reported PostgreSQL bugs, nine fixed upstream (six with our patches), with the tool and all data (Section 8).

2 PROBLEM STATEMENT

Let G be a typed grammar for the SQL dialect of PostgreSQL and S a database schema. A *test case* is a sequence of statements derived from G over S, run in one transaction that is rolled back at the end. The *derivation tree* $T(Q)$ of a query Q records the alternative each rule took; its leaves, read left to right, are the text of Q. A query has a *wrong result* when the server returns, without an error, a multiset $[\![Q]\!]$ that differs from what the SQL standard and the PostgreSQL documentation define. An *oracle* maps the server's response to a test case to *pass* or *bug candidate*: a metamorphic oracle such as TLP [Rigger and Su 2020b] or NoREC [Rigger and Su 2020a] compares Q with a rewritten query, and a *plan variant* (the same query under a setting such as `SET enable_hashjoin = off`) must return the same multiset.

Sound rewrites. A tree rewrite r maps $T(Q)$ to a second tree. It is *sound* for Q if $[\![Q]\!] = [\![r(Q)]\!]$ holds on every correct server. A rewrite that is sound only under conditions (for example, only if no `LIMIT` without `ORDER BY` appears below the rewritten node) must check them on the tree before it is applied. An unsound rewrite produces *false positives*: candidates that are not bugs.

Goal. Given a fixed time budget B and one server, find as many distinct bugs as possible, where a bug is a crash, a failed assertion, an internal error (SQLSTATE class `XX`), or a wrong result, while keeping the *precision* of each oracle high:

$$precision(o) = \frac{|R_o|}{|R_o| + |F_o|} \tag{1}$$

where R_o and F_o are the real bugs and the false positives among the distinct candidates oracle o reported, after de-duplication by signature (Section 3.4). Candidates that do not reproduce (flaky) are counted apart, and a duplicate of a bug reported upstream is a real bug that we report separately, so we also give precision with duplicates in R_o.

Why not let an LLM write the test cases? An LLM can write SQL that targets a feature, but at a few seconds per call it writes at most a few thousand test cases in an hour, where the grammar arm produced 71,228 (Table 1). LLM-written queries are also not derivation trees, so the tree-based oracles cannot rewrite them. We use the LLM to extend the grammar instead: a few calls per commit, after which the grammar generates thousands of variations.

3 APPROACH

Figure 2 shows the steps of Arbor. The main idea is to keep every generated query as a typed derivation tree until it has been run, so that oracles can rewrite single nodes of known type and position instead of SQL text. The tree has four jobs: generation builds it, mutation re-derives one subtree, the oracles rewrite one typed node, and triage reduces it by replacing subtrees with their smallest derivation. The fuzzing loop (Steps 1–4) runs thousands of test cases per minute; the LLM loop (Step 5) runs a few times per commit, before the campaign.

3.1 Step 1: Derive Test Cases from a Typed Grammar

We wrote a grammar for the SQL that PostgreSQL 18 and master accept, in a small format of our own (`.pgg`): 78 rules with 421 alternatives over 13 SQL types. A rule may be parameterised by a type, as in `expr<T>`, so a derivation only combines expressions that type-check. Weights set how often each alternative is chosen, and *builtins* written in Python handle what a context-free grammar cannot: `@col(T)` picks a visible column of type T in the current

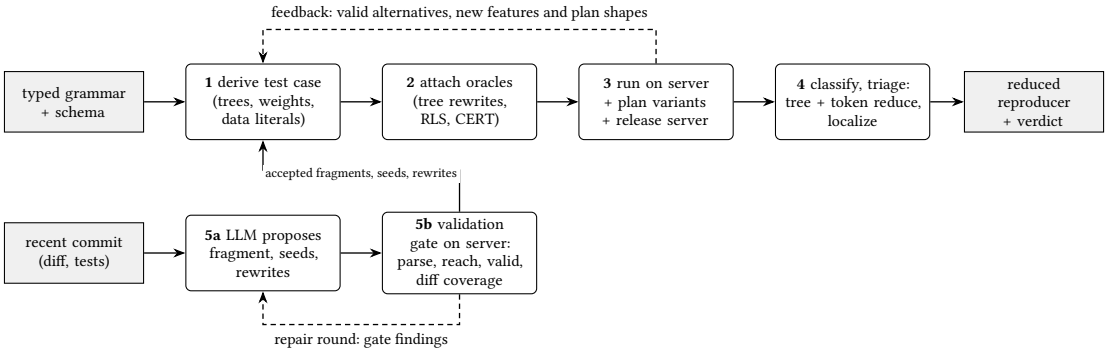


Figure 2: Overview of Arbor. The top row is the fuzzing loop: test cases are derived from the typed grammar, oracles are attached as rewrites of the derivation tree, and the server runs each query under several settings. The bottom row runs before a campaign: an LLM turns a recent commit into grammar alternatives, and the validation gate accepts them only when the server runs them and they reach the changed lines.

scope, `@table` binds a fresh alias, and `@group_keys` makes the current query level grouped, after which only grouped columns and aggregates are visible. The schema has 9 relations with partitions, inheritance, a view, indexes of four access methods and NULLs in most columns.

A test case has up to two setup statements (DDL, DML, `ANALYZE`, `SET LOCAL` of a planner setting) followed by one or two queries. While deriving, the engine wraps the tokens of every node in invisible markers, so after rendering it knows the exact token span of each node. Half of the test cases are new derivations; the others replay a corpus test case and re-derive one subtree. A test case enters the corpus when it produces a new bug signature, a new SQL feature, a new pair of features, or a new plan shape, the digest of the node types in `EXPLAIN` output [Ba and Rigger 2023].

Feedback and data. Every alternative carries a multiplicative weight that grows when a test case that used it enters the corpus and shrinks when its statement fails (rate $\eta = 0.1$, range $[0.2, 5]$). Some literals come from a *data profile* (minimum, maximum and quantiles of every column, type edges such as -0, NaN, ±Infinity and the empty array), so predicates select a real share of rows. Grammar-based generation is not new; the typing, scoping builtins and node spans are what the later steps need.

3.2 Step 2: Attach Oracles as Tree Rewrites

For each query, Arbor picks up to one metamorphic check and stores it with the test case as a pair of queries that must return the same multiset. All checks are built on the derivation tree.

Partitioning and counting. TLP on `WHERE` picks a `SELECT` whose result is used as a multiset (the query itself, a derived table, a CTE body, a set-operation arm, or an `IN`, `EXISTS` or `LATERAL` subquery) and partitions its `WHERE p` into p, `NOT p` and `p IS NULL`, combined with `UNION ALL`. Because the rewrite edits one node, it also works in a nested query, where text-based TLP does not apply. *TLP on* `HAVING` does the same; the unpartitioned side keeps `HAVING TRUE`, because without `GROUP BY` a `HAVING` clause is what makes a level a single-group aggregate. *TLP on aggregates* equates a global `MIN`, `MAX`, `SUM`, `COUNT`, `BIT_*` or `BOOL_*` with the combination over the three partitions (the `MIN` of the `MIN`s, the `SUM` of the `COUNT`s). *NoREC* compares `count(*)` with `WHERE p` against the sum of `CASE WHEN p THEN 1 ELSE 0 END`.

Equivalent forms of one node. Expression rewrites replace one typed expression e by `COALESCE(e,e)`, `(SELECT e)`, `CASE WHEN TRUE THEN e END`, $e+0$ or $e \cdot 1$ (numbers), `e || ''` (text), or `NOT NOT p`, `p AND TRUE`, `p OR FALSE` (predicates). *Predicate rewrites* turn `x IN (a, b)` into `(x = a) OR (x = b)`, which PostgreSQL 18 turns back into `= ANY`, `a < b` into `NOT (a >= b)`, which three-valued logic keeps exact, and `BETWEEN` into two comparisons. *NOT NULL conjunct* turns `WHERE p` into `WHERE p AND c IS NOT NULL` for a NOT NULL column c that p uses, which targets the planner's NOT NULL simplifications; it is skipped under outer joins and for generated columns, where such a column can still be NULL. *ORDER BY removal* drops the statement's own `ORDER BY`, *CTE materialisation* toggles `MATERIALIZED`, and the *table fence* turns one table reference into `(SELECT * FROM t OFFSET 0)`, which stops the planner from pushing predicates into it, removing it, or pruning its partitions.

Security and estimates. For *row-level security*, five tables carry a policy defined in one file used by both the server and the rewrite. The query runs as a restricted role, under the policies, and as a superuser with every protected table t replaced by `(SELECT * FROM t WHERE policy)`; a difference means that the policies let a row through or hide one they should not. Following CERT [Ba and Rigger 2024], adding the `WHERE` clause to a query must not raise the planner's row estimate; this check compares `EXPLAIN` estimates, and a violation is a *performance* candidate, reported apart from bugs.

A check is attached only if its rewrite is sound for that query (Section 2).

3.3 Step 3: Run the Test Case Several Ways

The server runs each test case in a transaction with one savepoint per statement and rolls it back at the end, so test cases cannot affect each other. For each deterministic query Arbor also runs (i) both sides of the attached check; (ii) the query under two of 18 *plan*

variants (for example hash join only, no hash aggregation, forced parallel plans, partitionwise join, no eager aggregation, JIT), chosen at random per query; and (iii) the whole test case on a server of the last release, PostgreSQL 18.6, in a database of its own for each worker (the *cross-version oracle*). Results are compared as order-insensitive digests that drop choices no plan controls: trailing zeros of equal `numeric` values, the sign of float zero, the last digits of finite floats in aggregates, and the text of equal `interval` values.

3.4 Step 4: Classify, De-duplicate, Triage

The oracle reads the statement results and the server log. A `TRAP` line is an assertion failure; a lost connection without one is a crash; an error of class `XX` is an internal error; a confirmed mismatch of any digest is a wrong result (a metamorphic mismatch counts only if it repeats when both sides run again). Crash evidence counts only if it carries the process id of the test case's own backend or its parallel workers. Each candidate gets a *signature*: the assertion location, the normalised backtrace or error message, or, for wrong results, the oracle and the shape of the query; only the first test case per signature is saved.

Triage reproduces the candidate three times on a fresh connection and reduces it in two stages. *Tree reduction* works on the derivation trees that every failure of the grammar arm stores: it drops a statement, drops one repetition, or replaces a subtree by the smallest derivation of the same rule and type, and replays the trees through the generator, which re-binds names, aliases and types. Every candidate is therefore valid SQL, and the server's time goes to the question whether the failure remains. *Token reduction* follows: delta debugging [Zeller and Hildebrandt 2002] removes statements, then clauses, list items and parenthesised groups level by level over the parenthesis tree [Misherghi and Su 2006], then single tokens. A step is kept only if the same failure occurs (same assertion, normalised message, or oracle and plan variant); for a metamorphic candidate both queries are reduced together, so they stay a valid pair.

Localization. For a reduced wrong result, Arbor runs the queries under every plan variant. If a variant removes the disagreement, the bug is in the code path that setting switches, and the report names the setting and its source files; if none does, the cause is an always-applied rewrite, constant folding, a planner support function, or expression evaluation. The output is a short reproducer, the rows each side returned, the localization and a draft report in the shape of the PostgreSQL bug form.

3.5 Step 5: Extend the Grammar from Recent Commits

Step 5a: propose. Arbor lists the commits of PostgreSQL master that change the optimiser, executor, parser or SQL-callable functions and add regression tests. For each it builds a digest: the message, the changed functions, the SQL the commit added to the tests, and the *diff-coverage targets*, the changed lines of each C file. An LLM gets the digest with a fixed, cached system prompt (the `.pgg` format, the builtins, the conventions of the grammar, the schema and the whole current grammar) and answers with structured output: a rationale, a fragment of new alternatives, one to five seed test cases, the features it targets, and up to three *equivalence rewrites*, templates over one expression of a given type such as `CASE WHEN {e} IS NULL THEN NULL ELSE {e} END` that the model claims equal `{e}` for every value. The commit miner never shows the model a fix that the benchmark of Section 4.6 reverts.

Step 5b: validate. No model output reaches the campaign unless the server agrees. The validation gate checks in order: (i) *parse*: the fragment parses with the grammar, adds no types and uses only known builtins; (ii) *reachability*: in a sample of 150 test cases with the new alternatives boosted, they are used at least five times; (iii) *validity*: at least 60% of the statements that use them run without error; (iv) *seeds*: each seed runs; and (v) *diff coverage*: on a gcov build, the sample executes more of the commit's changed lines than the same sample without the fragment. A failed check sends its findings (the parse error, the most frequent error messages, the uncovered lines) back to the model for up to three repair rounds; every round is a ledger line with its tokens, cost and the gate's report.

A proposed rewrite is a claim about semantics, so the gate tests it twice. The server evaluates `rewrite(v) IS NOT DISTINCT FROM v` for NULL, the edge values of the type and quantiles of the data; then Arbor applies the rewrite alone to the expressions of 150 test cases, with confirmation re-runs. The rewrite is accepted only if no value and no query shows a difference, and it then becomes an expression rewrite of Step 2. Neither test varies the collation, so the gate also rejects a template with an explicit `COLLATE` (Section 4.7).

4 EVALUATION

We ask four questions.

- **RQ1** Does the typed grammar reach more of PostgreSQL than a random-mutation generator in the same time: more valid statements, more covered lines and branches, more SQL features and plan shapes?
- **RQ2** Which bugs does Arbor find, and how precise is each oracle?
- **RQ3** On real bugs put back into PostgreSQL, which configuration finds how many, and how fast?
- **RQ4** What does the commit-directed LLM extension add, and what does it cost in tokens, money and repair rounds?

4.1 Subject and Benchmark

All servers are built from PostgreSQL master `26bd08fb` (16 October 2026) with assertions on. For **RQ3** we built a benchmark in the style of Magma [Hazimeh et al. 2020] by reverting fixes: every commit between 31 July and 16 October 2026 that fixes a bug in the optimiser, executor, parser, rewriter or a SQL-callable function, adds a regression test, and reverts cleanly alone and together with the others, 23 in all. We build the *fixed* server, one *solo* server per bug (only that fix reverted), and the *injected* server (all 23 reverted) on which the campaigns run. A proof of concept (PoC) written from each regression test changes its result for 20 bugs: 8 wrong results, 5 internal errors, 4 assertion failures, 2 missing errors and 1 wrong error. A crash-only fuzzer can see only the 9 assertions and internal errors. The other three change only the plan, or need an equality operator that returns NULL for non-NULL inputs, and leave the denominator. On the injected server every PoC shows the symptom of its solo server.

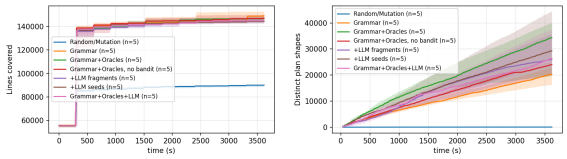


Figure 3: RQ1: lines covered (left) and distinct plan shapes (right) over time, median and interquartile band of 5 trials per arm.

4.2 Configurations

The configurations (*arms*) share server, schema, executor, oracles, budget and per-trial seeds and differ only in the generator. Random mutates seed queries at the token level and attaches no metamorphic checks. Grammar is Step 1 alone. Grammar+Oracles adds Steps 2–4: metamorphic checks (including RLS and CERT) on half the queries, plan-shape guidance, feedback weights and data-profile literals; Grammar+Oracles, flat is the same without the weights. +LLM adds the fragments, seeds and rewrites that passed the validation gate, and +LLM fragments and +LLM seeds use one of them. The commit miner excludes the 23 benchmark fixes. All arms use the plan-variant oracle and, in **RQ3**, the cross-version oracle.

4.3 Setup

Arbor is about 10,500 lines of Python with 151 unit tests; it drives PostgreSQL through `psycopg` and reads the server log for assertion and crash evidence. Each arm runs 5 one-hour trials with four workers, one server each. **RQ1**, **RQ2** and **RQ4** ran on one laptop (WSL2, 24 logical cores), **RQ3** on another, so each experiment is compared on one machine. The cross-version reference is PostgreSQL 18.6. Following Klees et al. [Klees et al. 2018], and Arcuri and Briand [Arcuri and Briand 2011], we report medians, compare arms with the two-sided Mann–Whitney U test, and give the Vargha–Delaney $\hat{A}_{12}$ effect size [Vargha and Delaney 2000]. With 5 trials per arm the smallest possible two-sided p is 0.008, so we read effect sizes, not only p. Coverage is sampled every 300 seconds per worker, so the first point of every curve is the server's start-up coverage.

The language model. In the client's manual mode a script answered the prompts with Google Gemini Flash models on the free tier (9 answers by `gemini-3.6-flash`, 2 by `gemini-3.8-flash`, 14 by `gemini-3-flash-preview`); the daily quota limited **RQ4** to 10 of the 25 mined commits. The results of **RQ4** hold for these models.

4.4 RQ1: Reach

Table 1 gives the medians of the seven arms and Figure 3 the growth over time.

Observation 1. The typed grammar reaches far more than random mutation: 36.7% against 22.2% of lines ($\hat{A}_{12} = 1.00$, $p = 0.008$), 101 against 23 SQL features, and 99.3% against 26.0% valid statements, although the random arm runs 15 times more test cases.

Observation 2. The oracles do not add line coverage (36.7% to 36.2%, $p = 0.42$) but they roughly double the plan shapes (20,279 to 34,313, $\hat{A}_{12} = 0.84$, $p = 0.095$, so with 5 trials this is an effect size, not a significant difference). The feedback weights contribute: without them the same arm reaches 23,997 plan shapes ($\hat{A}_{12} = 0.80$, $p = 0.15$), at the price of validity (96.3% against 99.4%, $p = 0.016$).

Oracles also cost throughput: the plan-variant runs and metamorphic checks lower the statements per second from 67 to 40.

4.5 RQ2: Bugs and Oracle Precision

Across the **RQ1** and **RQ3** campaigns, plus the earlier hunt and smoke runs, Arbor reported 93 distinct bug candidates (after deduplication by signature). We triaged all of them (Section 3.4) and Table 2 gives the verdicts per oracle.

Bugs. Table 3 lists the eleven bugs we reported. Seven came from running the fuzzer (the campaigns and later runs on newer master builds); four came from the targeted bug hunt that accompanied them, in which we audited recent planner and executor features with differential probes. Nine of the eleven bugs are fixed upstream, six of them with fixes written by the authors; #19828 was fixed by a committer's patch for a separate report of the same error. The other two are open with our patches. The oracles also found three bugs reported by others days before us, which we count as duplicates: a wrong result of self-join elimination under `ROLLUP` in 18.6 (#19800, a plan variant), #19772 (the `(SELECT e)` rewrite) and #19784.

Rediscovery. We added each reproducer as a seed that carries its own check (for a wrong result, the query that must agree), extended the grammar with what the four hunt bugs need (statistics on expressions, statistics import with boundary values, a user-defined implicit cast, `IGNORE NULLS` over whole-partition frames), and added a *hang* oracle that restarts a server whose statement outlives its `statement_timeout` by ten seconds. One 150-second trial of Grammar+Oracles on the base build then reported all nine bugs reported up to that point, each matched to its catalogue entry, at unchanged validity (99.3%). With the tenth reproducer added (#19826, found by the plan-variant oracle under partitionwise aggregation), the one-command reproduction, one 150-second trial that replays the seeds in turn, reports all ten bugs that exist in the base build. The eleventh, #19828, was introduced and fixed on master within one day after the base build; its reproducer raises the internal error on a build from that window (`REPRO_19828=1 scripts/reproduce.sh`).

Observation 3. Plan-variant and metamorphic oracles found real wrong results in released versions that no crash oracle can see (BUG #19793 and #19800).

Precision. Of the 93 candidates, 28 (30%) did not reproduce in three re-runs. Of the 65 others, 8 are real, 15 are duplicates of real bugs and 42 are false positives, so the precision is $8/50 = 0.16$ without duplicates and $23/65 = 0.35$ if duplicates count as true positives. The false positives come from few classes: six were fixed before the campaigns, and five that the campaigns revealed were fixed *after* them, tested on the stored reproducers but not re-measured. They are equal `interval` values printed differently depending on

Table 1: RQ1: median of 5 one-hour trials per arm on an instrumented PostgreSQL master. Valid is the share of statements that run without error; lines and branches are of the backend (gcov); features are of a catalogue of 148.

Arm	Valid (%)	Lines (%)	Branches (%)	Features	Plan shapes	Test cases
Random	26.0	22.2	15.1	23	43	1,074,240
Grammar	99.3	36.7	27.1	101	20,279	71,228
Grammar+Oracles	96.3	36.2	26.7	101	34,313	58,601
Grammar+Oracles, flat	99.4	36.2	26.8	101	23,997	64,923
+LLM fragments	95.5	35.7	26.5	104	25,972	48,222
+LLM seeds	98.3	35.6	26.4	106	29,274	63,655
+LLM (both)	95.3	35.7	26.4	106	26,375	46,754

Table 2: RQ2: verdict of every triaged candidate per oracle. Duplicate: a real bug that had already been reported upstream. False positives include one intended change between versions. Flaky: did not reproduce in three re-runs. Open: not decided yet.

Oracle	Candidates	Real	Duplicate	False pos.	Flaky	Open
assertion	15	3	10	1	1	0
crash	1	0	0	1	0	0
cross-version	25	0	0	7	18	0
internal_error	4	2	1	0	1	0
mm: expr	3	1	1	1	0	0
mm: rls	5	0	0	5	0	0
mm: tlp_having	4	0	0	4	0	0
mm: tlp_where	4	1	1	2	0	0
plan-variant	32	1	2	21	8	0
all	93	8	15	42	28	0

Table 3: The eleven bugs we reported upstream, the oracle that recognises each, and their state on 13 November 2026. All but #19828 reproduce on the base build `26bd08fb`, which predates the fixes; #19828 is a regression of a later master commit and was found by a run on newer master.

Bug	Report	Oracle	State
`JSON_TABLE` assigns no collations	#19768	assertion	fixed (our patch)
`range @> NULL` folded to a constant	#19793	`(SELECT e)` rewrite	reported, patch
`MERGE`: SubPlan with no parent	#19802	internal error	fixed
parallel `Append` assertion	#19803	TLP (+LLM fragments)	fixed
`to_char(NaN)` breaks `NORMALIZE`	#19771	assertion	fixed (our patch)
`NOT IN` anti-join ignores a cast	pgsql-bugs	TLP	fixed (our patch)
`IGNORE NULLS` frame never returns	pgsql-bugs	hang	fixed (our patch)
duplicate statistics expression	pgsql-bugs	assertion	fixed (our patch)
non-finite imported MCV frequencies	pgsql-hackers	CERT estimate	patch
`corr()` of a constant under partial aggregation	#19826	plan variant	fixed (our patch)
Memoize: subplan variable in a sublink	#19828	internal error	fixed

join order (16 candidates; the digest now uses PostgreSQL's interval equality), order-sensitive aggregates sorted by a key that is not the value, such as `ORDER BY NULL` (2), state that diverges in the cross-version oracle after an `INSERT` only master accepts (5), an RLS rewrite that left a protected table inside a repeated subselect unfiltered (5), and an LLM rewrite that is not an identity (Section 4.7). One remaining candidate is a new bug, which we reported as BUG #19826: `corr()` of a constant input is NULL serially but 1 under partial aggregation. It is fixed upstream with our patch (`ef1b7404`, back-patched to 19).

4.6 RQ3: Re-injected Bugs

Observation 4. With one hour and five trials, the arms found few of the benchmark bugs: Random 0 of 20, Grammar 1, Grammar+Oracles 2 and +LLM 2. No arm found any of the 8 wrong-result bugs.

The grammar arms found the internal error of fix `dfcf6fa7` in nearly every trial (median 458, 509 and 1,109 seconds); Random never did. The two arms with oracles also found the assertion of `11b56ee2`, once each, which is too little to separate them. The benchmark therefore does *not* show an advantage of the oracles; that claim rests on **RQ2**. Three things limit **RQ3**: one hour is short for bugs that need a specific combination of features; attribution is conservative (of the 30 wrong-result failures, 11 reproduce on the

Table 4: RQ3: trials (of 5) in which an arm found each re-injected bug, with the median time to the first finding. The rows below list, per symptom, how many bugs an arm found at all. Only bugs found by some arm have their own row; 18 of 20 were found by none.

Bug	Symptom	Random	Grammar	Grammar+Oracles	+LLM (both)
11b56ee2	assertion	0/5	0/5	1/5 (2901 s)	1/5 (1858 s)
dfcf6fa7	internal error	0/5	5/5 (458 s)	4/5 (509 s)	5/5 (1109 s)
wrong result, 8 bugs: found		0	0	0	0
internal error, 5 bugs: found		0	1	1	1
assertion, 4 bugs: found		0	0	1	1
missing error, 2 bugs: found		0	0	0	0
wrong error, 1 bug: found		0	0	0	0
Bugs found (of 20)		0	1	2	2

fixed server, 19 on no server, and none on a solo server); and five trials cannot separate arms that find a bug in one trial in five.

4.7 RQ4: The LLM Extension

The gate accepted 4 of 10 commits (2 in the first round, 2 after repair). Of the 25 rounds, 16 failed because too few statements ran (mean validity of a proposal 36.5%), 4 because the fragment was not derived or did not parse, and 1 on seeds or coverage. The run used 328,000 input and 171,000 output tokens at no cost on the free tier.

Observation 5. The LLM arms add SQL features (101 to 104–106, $\hat{A}_{12} = 1.00$, $p = 0.004$) but no coverage: lines drop from 36.2% to 35.7% ($p = 0.42$), plan shapes from 34,313 to 26,375, and validity from 96.3% to 95.3%.

It covers only 34 more changed lines over the 10 commits, but the fragments-only arm found BUG #19803: with these models the benefit is modest.

The gate is not enough for rewrites. All 7 proposed rewrites passed the gate, but `NULLIF({e}, NULL::text COLLATE "POSIX")` is not an identity: it changes the collation, and caused a false positive in **RQ3**. Neither the value probe nor the live checks vary the collation; the gate now rejects an explicit `COLLATE`. Testing an equivalence on samples is not proof.

5 DISCUSSION

Is a larger grammar always better? No. A larger grammar reaches more corner cases of SQL semantics, and some break the assumptions of an oracle: `HAVING` without `GROUP BY`, `DISTINCT` over `numeric` values of different scale, `interval` values that are equal but print differently. The cost is paid in oracle precision, and every rewrite needs a soundness check on the tree.

Why keep the tree? Mutation re-derives a subtree, the oracles rewrite one typed node, and reduction replaces a subtree by its smallest derivation; each time the generator replays the tree and re-binds names and types, so the result stays valid SQL. On the `JSON_TABLE` case, tree reduction followed by token reduction gave a 79-token reproducer where token reduction alone gave 116. On the tree a clause is one node with a known span, which lets Arbor apply TLP inside a derived table or a `LATERAL` subquery, where a text rewrite must parse the query again.

Why compare with the last release? Plan variants cannot catch a wrong result that every plan shares, such as a constant folded before any plan is chosen (Figure 1). The release lacks the newest planner rules, so it disagrees with exactly the bugs that new code brings, and also with intended changes, which triage must recognise.

Can an LLM be trusted? Not on its word. A rewrite is a claim about semantics, and **RQ4** shows our gate accepted a wrong one. For fragments the model only has to name the SQL that reaches a commit, and the validation gate rejects what does not parse, is never derived, fails on the server or misses the changed lines. Repair helps rarely: of the 4 accepted commits 2 passed in the first round, and 6 of 10 never passed.

Does the approach carry over to other engines? Yes, with a small adapter per engine. After the PostgreSQL campaigns we generalized Arbor (public branch `generalized`) to DuckDB, SQLite, Turso and Apache DataFusion. Each engine runs in its own worker process, and a dialect *overlay* of a few dozen lines drops or rewrites the parts of the PostgreSQL grammar the engine does not accept. With the manual overlays, 96.3% of the generated queries are valid on DuckDB, 97.7% on SQLite, 93.3% on Turso and 79.4% on DataFusion. An overlay miner drafts the overlay from the measured failure rate of each grammar alternative: one mined round lifts DuckDB from 32.1% to 44.1% valid statements without any manual line. Optimizer-rule firing, read from rule switches or from per-rule plan traces, replaces the PostgreSQL plan variants as feedback and as a coverage measure. Other additions are Z3-synthesized rows that make TLP partitions non-empty, normalizers learned from probes for cross-engine comparison (DuckDB against SQLite: held-out disagreements drop from 28.7% to 3.6%), engine test suites as seeds, and a loop that triages new failures and drafts a filing package with a regression test in the engine's own format. *Rule kernels*, minimal queries that each make one optimizer rule fire, composed in pairs, raise the number of distinct optimizer-rule pairs reached in 15-minute campaigns (medians of three trials per arm) from 307 to 386 on DuckDB, 241 to 276 on DataFusion and 86 to 182 on SQLite; every kernel trial beat every plain one ($\hat{A}_{12} = 1.00$), although three trials per arm cannot give $p < 0.1$. The campaigns found fourteen bugs. Nine were already fixed on the engine's development branch when we checked. For the other five we filed an issue and a pull request with a fix and a regression test: Turso #11594 / PR #11595

(`percentile_disc` with an expression argument) and #11796 / PR #11797 (an aggregate in `IN (SELECT ...)` next to a window function: a spurious error, or a panic), DuckDB #28551 / PR #28552 (`min`/`max` over a nested value computed by a subquery, a 2.0.0 regression; approved by a maintainer for merging and for the 2.0 release line), DataFusion #27235 / PR #27236 (`UPDATE`/`DELETE` that repeat an expression fail on an internal common-subexpression column), and a DataFusion internal error for a scalar subquery inside `VALUES`. One command (`scripts/reproduce_generalized.sh`) reproduces all five on the released versions the campaigns ran on.

6 THREATS TO VALIDITY

Internal threats. Every arm runs 5 trials with the same per-trial seeds on the same build, schema, executor, oracles and budget, each trial has its own server, a metamorphic mismatch counts only if it repeats, and every candidate is reproduced three times before triage. But five trials are few: differences with p above 0.008 are effect sizes, not significant results, and several conclusions rest on them.

Our own oracles can be wrong. Eleven false-positive classes reached triage, six before the campaigns and five in them (Section 4.5). We fixed the last five *after* the campaigns and checked them on the stored reproducers and with unit tests, but did not re-run the campaigns, so the precision after the fixes is not a measurement. More classes may remain. Verdicts are ours, some made from the reduced reproducer; we release the reproducers. An accepted LLM rewrite is tested, not proven (Section 4.7). Attribution in **RQ3** replays on single-bug servers, so a finding that needs two injected bugs is reported as unattributed.

The two laptops differ, so **RQ1** and **RQ3** are not comparable in absolute throughput; 14 **RQ1** trials were re-run with the same seeds after a server failed crash recovery.

External threats. We test one system at one commit, with one schema of 9 relations and one set of row-level security policies that we chose; bugs that need other data shapes, extensions or concurrent sessions are out of reach. The 23 benchmark bugs are recent fixes with regression tests and may favour features that recent PostgreSQL development touched. The LLM results hold for free-tier Gemini Flash models on 10 commits; we store every prompt and response.

Construct threats. Coverage measures what code ran, not whether an oracle could have seen a bug there. A bug "found" in **RQ3** means a test case failed on its solo server and passed on the fixed one, even if its symptom differs from the PoC's. A fragment that passes the validation gate reaches the changed lines; that does not prove it tests the changed behaviour. The rediscovery of the reported bugs replays their reproducers as seeds: it shows that the oracles recognise each bug, not that generation alone finds it in a short run.

7 RELATED WORK

Metamorphic oracles for database systems. Closest to us is the work of Rigger and Su: pivoted query synthesis [Rigger and Su 2020c], NoREC [Rigger and Su 2020a] and TLP [Rigger and Su 2020b] in SQLancer, and equivalent expression transformation by Jiang and Su [Jiang and Su 2024]. These tools found hundreds of bugs, but they build the follow-up query from strings that their own small generators produced. We share their oracles and apply them to typed derivation trees of a grammar that covers much more of PostgreSQL, at any query level, and we report the precision of each oracle on that larger language.

Guidance for database fuzzing. Query plan guidance [Ba and Rigger 2023] steers SQLancer toward new plan shapes; SQUIRREL [Zhong et al. 2020] mutates an intermediate representation of SQL with coverage feedback to find crashes, and SQLRight [Liang et al. 2022] adds the logic-bug oracles of SQLancer. CERT [Ba and Rigger 2024] checks that a more restrictive query never gets a larger cardinality estimate; we include its check. Directed greybox fuzzing [Böhme et al. 2017] steers toward chosen code. We direct the campaign toward recent commits at the level of SQL features rather than branches.

Grammar-based and differential SQL testing. RAGS [Slutz 1998] compared the results of several database systems on random SQL, and SQLsmith [Seltenreich [n. d.]] generates queries from the catalog of a running PostgreSQL. We compare one server with itself under 18 plan variants and with its last release, which avoids most dialect differences, and we need a typed grammar because the oracles must know the type and scope of every node.

LLMs for fuzzing. Fuzz4All [Xia et al. 2024] uses an LLM as the input generator; we call it a few times per commit for grammar alternatives and rewrites, behind a server-side validation gate.

Benchmarks, evaluation and reduction. Magma [Hazimeh et al. 2020] ports real bugs into real programs, and Klees et al. [Klees et al. 2018] showed how fuzzing evaluations go wrong without repeated trials; our benchmark reverts real fixes in one server, attributes each finding by replay on single-bug servers, and every arm runs repeated trials. We reduce failures with delta debugging [Zeller and Hildebrandt 2002] and hierarchical delta debugging [Misherghi and Su 2006] and add paired reduction for metamorphic checks, where the two queries must shrink together to stay equivalent.

8 CONCLUSION

Arbor generates SQL as typed derivation trees, rewrites the trees into metamorphic checks, compares each query with its plan variants and the last release, and extends its grammar from recent commits behind a server-side gate. We reported eleven PostgreSQL bugs found with it, nine of them fixed upstream (six with our patches). Its grammar reaches 36.7% of backend lines against 22.2% for random mutation, yet the precision of the oracles, not the reach of the generator, decided how useful its findings were. With a dialect overlay and a small adapter per engine, the same core tests DuckDB, SQLite, Turso and DataFusion and found five more bugs there, which we reported upstream with fixes (Section 5).

DATA AVAILABILITY

The replication package is the public course repository https://github.com/PASS-IITG/cs5187-term-project-blazing-bhuvi (MIT license). The generalized fuzzer of Section 5 is on branch `generalized` of https://github.com/v28k/cs5187-term-project.

The `README.md` gives exact dependencies, a one-command reproduction of the headline result (`scripts/reproduce.sh`: the ten reported PostgreSQL bugs in the base build; `REPRO_19828=1` adds the eleventh), one for the five bugs of the generalized fuzzer (`scripts/reproduce_generalized.sh`), and every upstream report with its state. It contains the source of Arbor with its unit tests, the grammar and the schema. It also contains the benchmark: the 23 fix commits, their PoCs and the scripts that build the fixed, solo and injected servers. Finally, it holds the experiment configurations, the run logs and reports of every trial, the triage verdicts, and the LLM ledger with every prompt and response, so that **RQ4** can be replayed without calling the model.

USE OF GENERATIVE AI

Arbor calls a large language model as part of its method (Section 3.5). In the evaluation this was Google Gemini Flash (Section 4); every prompt and response, with the model that served it, is in the replication package. An AI coding assistant helped with parts of the implementation and with drafting text. The authors designed the approach, reviewed all code and text, and checked every statement, citation and number. All numbers reported here were produced by running the tool and scripts.

STATEMENT OF CONTRIBUTIONS

Every member can explain every part of the project; the split below records who did most of the work on each part. The commit history of the public repository shows the same split (GitHub users `v28k`, `Blazing-bhuvi` and `doma2542`).

Venu Kummari. Design and implementation of Arbor: the typed grammar and its generator, the metamorphic oracles on derivation trees, the plan-variant, cross-version, row-level-security and cardinality oracles, the executor and hang oracle, triage with tree and token reduction, the LLM gate, the re-injected bug benchmark and the evaluation harness. Reported BUG #19768 (`JSON_TABLE`) and BUG #19793 (`range @> NULL`) with patches, and BUG #19826 (`corr()` under partial aggregation, fixed upstream with his patch) and BUG #19828 (Memoize, fixed upstream). Wrote the paper. Built the generalized fuzzer for DuckDB, SQLite, Turso and DataFusion (Section 5) and wrote its five upstream fixes.

Bhuvan Chilukuti. The **RQ3** campaign on the re-injected bug benchmark and its attribution. Cross-version, metamorphic and audit-driven bug hunts on recent PostgreSQL features. Reported, with patches, the `NOT IN` anti-join bug, the `IGNORE NULLS` hang, the duplicate statistics expression assertion and the non-finite MCV frequencies; answered the maintainers on these threads.

Gowtham Reddy. Server builds, the **RQ1**, **RQ2** and **RQ4** campaigns (35 trials) with their reports, the LLM arm with Gemini Flash over ten commits, and the triage of the campaign candidates. Reported BUG #19802 (`MERGE`) and BUG #19803 (parallel `Append`), both fixed upstream (reporter and tester), and wrote the fix for BUG #19771 (`to_char`), committed upstream.

Signed, 9 November 2026: Venu Kummari Bhuvan Chilukuti Gowtham Reddy

REFERENCES

Andrea Arcuri and Lionel Briand. 2011. A Practical Guide for Using Statistical Tests to Assess Randomized Algorithms in Software Engineering. In *Proceedings of the 33rd International Conference on Software Engineering (ICSE)*. 1–10. https://doi.org/10.1145/1985793.1985795

Jinsheng Ba and Manuel Rigger. 2023. Testing Database Engines via Query Plan Guidance. In *Proceedings of the 45th IEEE/ACM International Conference on Software Engineering (ICSE)*. 2060–2071. DOI: verify.

Jinsheng Ba and Manuel Rigger. 2024. CERT: Finding Performance Issues in Database Systems Through the Lens of Cardinality Estimation. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*. 133:1–133:13.

Marcel Böhme, Van-Thuan Pham, Manh-Dung Nguyen, and Abhik Roychoudhury. 2017. Directed Greybox Fuzzing. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. https://doi.org/10.1145/3133956.3134020

Ahmad Hazimeh, Adrian Herrera, and Mathias Payer. 2020. Magma: A Ground-Truth Fuzzing Benchmark. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 4, 3, Article 49 (2020). https://doi.org/10.1145/3428334

Zu-Ming Jiang and Zhendong Su. 2024. Detecting Logic Bugs in Database Engines via Equivalent Expression Transformation. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 821–835.

George T. Klees, Andrew Ruef, Benjamin Cooper, Shiyi Wei, and Michael Hicks. 2018. Evaluating Fuzz Testing. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. https://doi.org/10.1145/3243734.3243804

Yu Liang, Song Liu, and Hong Hu. 2022. Detecting Logical Bugs of DBMS with Coverage-based Guidance. In *31st USENIX Security Symposium (USENIX Security)*. 4309–4326.

Ghassan Misherghi and Zhendong Su. 2006. HDD: Hierarchical Delta Debugging. In *Proceedings of the 28th International Conference on Software Engineering (ICSE)*. 142–151.

Manuel Rigger and Zhendong Su. 2020a. Detecting Optimization Bugs in Database Engines via Non-Optimizing Reference Engine Construction. In *Proceedings of the 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. https://doi.org/10.1145/3368089.3409710

Manuel Rigger and Zhendong Su. 2020b. Finding Bugs in Database Systems via Query Partitioning. *Proceedings of the ACM on Programming Languages* 4, OOPSLA (2020). https://doi.org/10.1145/3428279

Manuel Rigger and Zhendong Su. 2020c. Testing Database Engines via Pivoted Query Synthesis. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 667–682.

Andreas Seltenreich. [n. d.]. SQLsmith: A Random SQL Query Generator. https://github.com/anse1/sqlsmith. Accessed: 2026-10-23.

Donald R. Slutz. 1998. Massive Stochastic Testing of SQL. In *Proceedings of the 24th International Conference on Very Large Data Bases (VLDB)*. 618–622.

András Vargha and Harold D. Delaney. 2000. A Critique and Improvement of the CL Common Language Effect Size Statistics of McGraw and Wong. *Journal of Educational and Behavioral Statistics* 25, 2 (2000), 101–132. https://doi.org/10.3102/10769986025002101

Chunqiu Steven Xia, Matteo Paltenghi, Jia Le Tian, Michael Pradel, and Lingming Zhang. 2024. Fuzz4All: Universal Fuzzing with Large Language Models. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*. https://doi.org/10.1145/3597503.3639121

Andreas Zeller and Ralf Hildebrandt. 2002. Simplifying and Isolating Failure-Inducing Input. *IEEE Transactions on Software Engineering* 28, 2 (2002), 183–200.

Rui Zhong, Yongheng Chen, Hong Hu, Hangfan Zhang, Wenke Lee, and Dinghao Wu. 2020. SQUIRREL: Testing Database Management Systems with Language Validity and Coverage Feedback. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 955–970.